



汇编语言

6 算术运算类与位操作类指令

张青

ieqzhang@zzu.edu.cn

实验一 认识汇编语言

1. 准备实验环境：根据自己的电脑环境安装本书开发软件或登录希冀实验平台。
2. 录入教材例 1-1 程序 eg0101.c，编译生成目标文件 eg0101.o 和汇编语言文件 eg0101.s，用 objdump 反汇编 eg0101.o 观察汇编指令。
3. 录入教材例 1-2 程序 eg0102.c，编译连接生成目标文件 eg0102.o、汇编语言文件 eg0102.s 和可执行文件文件 eg0102，保证程序运行正确。用 objdump 反汇编 eg0102.o 和eg0102 观察汇编指令。
4. 模仿 eg0103.s，编写汇编语言程序 exp0104.s，输出字符串 “Hello Assembly! ” ，并汇编连接运行。

注意：

在自己电脑上安装实验环境可用来课下练习，所有实验必须在希冀平台（登录地址：**10.67.4.14**，登录账号：**学号**，默认密码：**123456**，已有账号的使用原密码）上完成，没有**在希冀平台上完成的实验，将不会记录实验成绩，影响最终成绩！！切记！！**

实验1 认识汇编语言

实验步骤1 实验目的

1. 熟悉 C 语言函数与汇编语言的关系
2. 掌握 GAS 汇编语句格式和程序框架
3. 掌握 GCC 的使用和 GAS 汇编语言开发过程。
4. 掌握 GCC 常用参数的使用。

▶ 下一个步骤

重要的文件建议保存在工作路径: `/mnt/cgshare` 下, 如果容器出现故障, 桌面还原之后, 点击“更多”/“工作目录文件浏览器”找回。



教学内容



- 复习传送类指令

教材章节：4.2

慕课：5.3

- 学习算术运算类指令

教材章节：4.3

慕课：5.4

- ▶ **MOV指令**

- ▶ **XCHG、LEA指令**

- ▶ **PUSH和POP指令**

- ▶ **状态标志**

- ▶ **算术运算指令**

学习目标



- 能正确书写算术运算类指令
- 能判断指令执行结果及对标志位的影响
- 能编写指令序列完成传送和算术运算
- 能写出C语言程序对应的汇编指令
- 能写出汇编指令序列对应的C语句
- 能用GDB调试程序
- 养成细致严谨的态度，注意C语言与汇编语言之间的关系，培养系统观

传送指令MOV



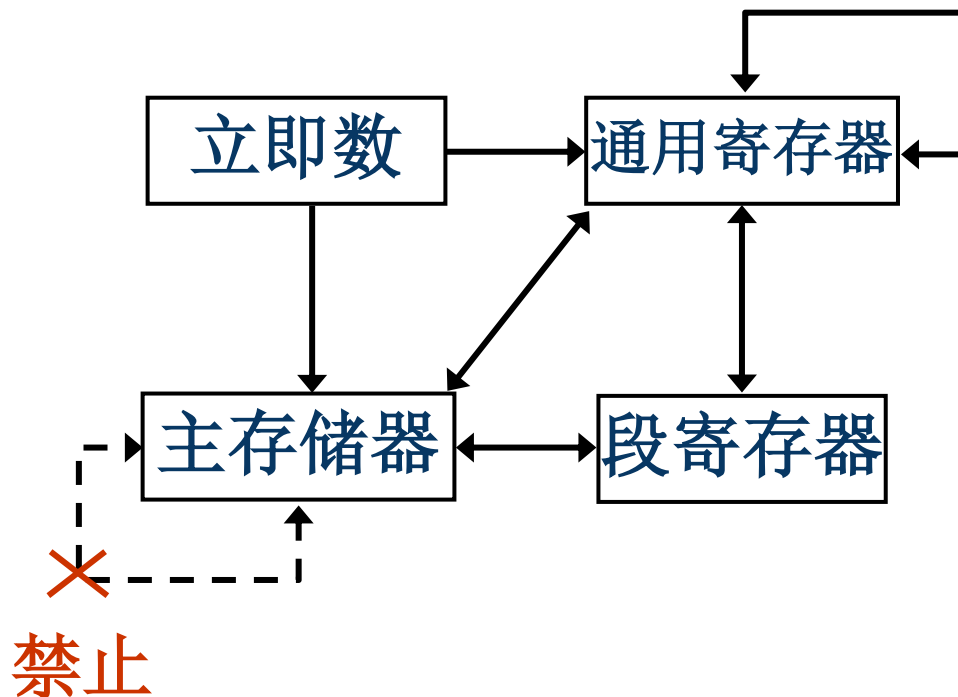
- 把一个字节、字、双字、四字的操作数从源位置传送至目的位置

MOV reg/mem,imm

MOV reg/mem/seg,reg

MOV reg/seg,mem

MOV r16/m16,seg



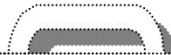
交换指令XCHG

- 将源操作数和目的操作数内容交换

XCHG reg,reg/mem

XCHG reg/mem,reg

- 通用寄存器与通用寄存器之间
 - 通用寄存器或存储器之间
- 空操作指令NOP (= XCHG EAX,EAX)
 - 处理器执行空操作该指令，需要花费时间，在主存中占用一个字节空间
 - 机器码：90
 - 实现短时间延时
 - 临时占用代码空间



```
xchg esi,edi  
xchg esi,[rdi]  
xchg al, var[rip]  
xchg rax,var[rip]
```

地址传送指令

- 地址传送指令获取存储器操作数的地址
LEA r16/r32/r64,mem
#r16/r32/r64←mem的有效地址EA (不需类型一致)
#r16:实地址模式, r32: IA-32e , r64: 64位模式
- LEA指令类似的地址操作符OFFSET的作用
 - LEA指令在指令执行时计算出偏移地址
 - OFFSET操作符在汇编阶段取得变量的偏移地址
 - OFFSET无需在执行时计算、指令执行速度更快
 - LEA指令能获取汇编阶段无法确定的偏移地址

lea rsi, var[rip] #64位模式推荐

mov rdi, offset var #仅用于intel语法, 不稳定, 避免使用

进栈指令PUSH



PUSH r16/m16/i16/seg

① $RSP = RSP - 2$ ② $SS:[RSP] = r16/m16/i16/seg$

PUSH r64/m64/i8/i32/i64

① $RSP = RSP - 8$ ② $SS:[RSP] = r64/m64/i8/i32/i64$

- 先将RSP减小作为当前栈顶
- 后将源操作数(立即数、通用寄存器和段寄存器内容或存储器操作数)传送到当前栈顶
- 以字或四字为单位操作
 - 进栈四字量数据时, RSP减8
 - 进栈字量数据时, RSP减2

push rax

出栈指令POP



POP r16/m16/seg

① r16/m16/seg = SS:[RSP] ② RSP = RSP + 2

POP r64/m64

① r64/m64 = SS:[RSP] ② RSP = RSP + 8

- 先将栈顶数据传送到目的操作数（通用寄存器、存储单元或段寄存器）
- 后ESP增加作为当前栈顶
- 以字或四字为单位操作
 - 出栈四字量数据时，RSP加8
 - 出栈字量数据时，RSP加2

pop rax

堆栈指令示例



#rax=0x0A

#假设rsp= 0x00000052C01FFDE8

push rax

rsp= 0x00000052C01FFDE0

pushf

rsp=0x00000052C01FFDD8

mov rcx,[rsp]

rcx=rflags=0x00000000000000202,

rsp=0x00000052C01FFDD8

mov rdx, [rsp+8]

rdx=rax=0x0000000000000000A,

rsp=0x00000052C01FFDD8

popf

rsp=0x00000052C01FFDE0

pop rax

rsp=0x00000052C01FFDE8

读写标志寄存器的值



RAX  **RFLAGS**

MOV RAX, RFLAGS
MOV RFLAGS, RAX

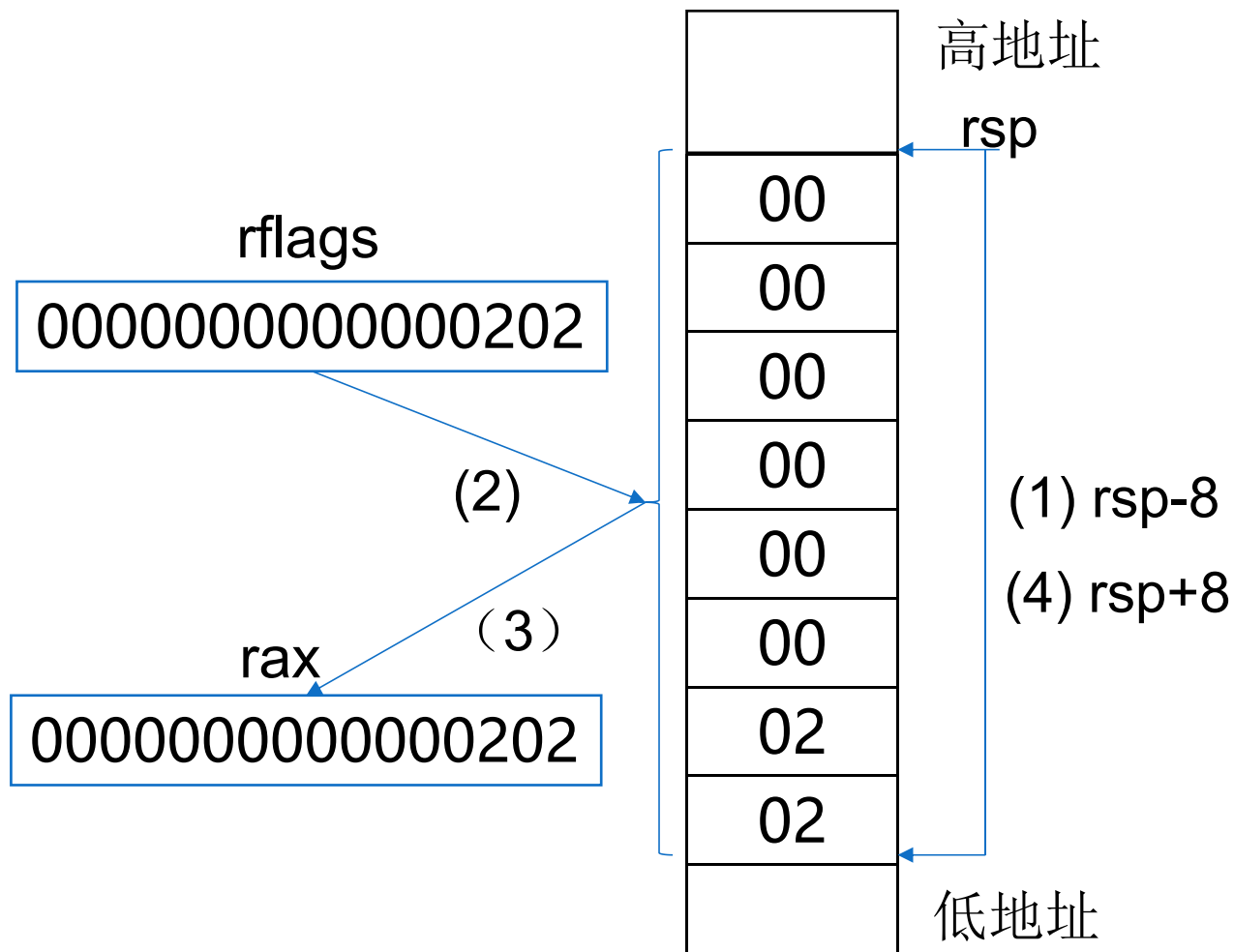


PUSH RAX
POPF

PUSHF
POP RAX

〔例4-3〕：查看标志寄存器的值

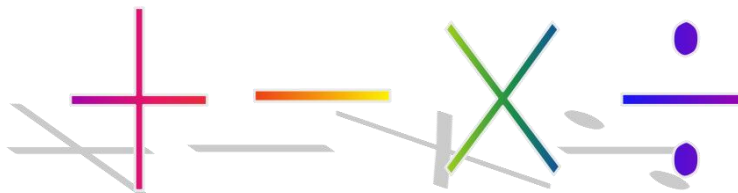
```
/*eg0403.S */  
.text  
pushf  
pop rax  
call disphx
```



- 拼写错误、多余的空格、不正确的标点、太过复杂的常量或表达式
- 操作数类型不匹配、无明确类型、错用（段）寄存器、两个存储器操作数、超出数据范围.....

算术运算类指令

- 算术运算
 - 对数据进行加减乘除
 - 基本的数据处理方法
 - 加减运算有“和”或“差”的结果外，还有进借位、溢出等状态标志，也是结果的一部分
- 注意算术运算类指令对标志的影响
 - 掌握：加法和减法指令
 - 熟悉：乘法和除法指令
 - 理解：零位扩展和符号扩展



加法指令



- 加法指令 **ADD**
- 带进位加法指令 **ADC**

ADD/ADC reg,imm/reg/mem #reg←reg+imm/reg/mem(+CF)

ADD/ADC mem,imm/reg #mem←mem+imm/reg(+CF)

- 增量指令 **INC**

INC reg/mem #加1: reg/mem←reg/mem+1

- INC不影响进位标志CF
- 其他指令按定义影响全部状态标志位

按照运算结果相应设置各个状态标志为0或为1

数据传送类指令**不影响**（=不改变）状态标志
加法和减法指令**根据结果按定义改变**状态标志

〔例4-4〕：add指令对标志位的影响



```
mov al,0x7
```

```
add al, 0xfb
```

```
#al=0x2, EFLAGS=0x00000213, OF=0, SF=0, ZF=0, AF=1, PF=0, CF=1
```

```
mov eax, 0x10000
```

```
add eax, 0x20000
```

```
#eax=0x30000, EFLAGS=0x00000206, OF=0, SF=0, ZF=0, AF=0, PF=1,  
CF=0
```

〔例4-5〕：C语言中的加法运算



```
//eg0405.c
#include <stdio.h>
int x=123456789, y=-978522123;
char c='A';
int main(){
printf("x+y =%d\n", x+y+250);
printf("c =%c\n",c+32);
return 0;
}
```



```
#x+y+250
mov     edx, DWORD PTR y[rip]
add     edx, DWORD PTR x[rip]
add     edx, 250
#c+32
movsx   edx, BYTE PTR c[rip]
add     edx, 32
```

〔例4-6〕：无符号64位二进制加法运算



```
.data
dvar1:  .long 0x82347856, 0x67783000
dvar2:  .long 0x12348998, 0x67762000
dvar3:  .quad 0
.text
lea r8, dvar1[rip]
lea r9, dvar2[rip]
lea r10, dvar3[rip]
```

〔例4-6〕：无符号64位二进制加法运算



```
mov eax,[r8]
add eax,[r9]      #低位双字相加
                  #eax=0x946901ee,EFLAGS=0x00000286

mov [r10], eax
mov eax,[r8+4]
adc eax,[r9+4]    #高位双字相加，再加CF，
                  #eax=0xceee5000,EFLAGS=00000A86

mov [r10+4], eax

mov rax,dvar3[rip]
call disphx
```

〔例4-7〕：修改例4-6的代码段



..... #与例4-6相同

.text

mov rbx,0

.....

mov eax,[r8+rbx*4]

add eax,[r9+rbx*4]

#低位双字相加,

#eax=0x946901ee,EFLAGS=0x00000286

mov [r10+rbx*4],eax

inc rbx

mov eax,[r8+rbx*4]

adc eax,[r9+rbx*4]

#高位双字相加, 再加CF,

#eax=0xceee5000,EFLAGS=00000A86

mov [r10+rbx*4],eax

mov rax,dvar3[rip]

课堂练习 (学习通)



- 下面C语言语句如何用汇编指令实现?

```
int x[10];  
x[0]=1;  
x[5]=x[0]+10;
```

```
x: .space 40  
mov dword ptr x[rip],1  
mov eax,x[rip]  
add eax,10  
mov x[rip+20],eax
```

- 下面汇编指令执行后, edx=?

```
lea edx, [rcx+rcx*4]  
lea edx, [rdx+rdx*2]
```

```
edx=5*rcx  
edx=5*rcx+5*rcx*2  
=15*rcx
```

减法指令



- 减法指令 SUB

- 带借位减法指令 SBB

SUB/SBB reg,imm/reg/mem #reg←reg-imm/reg/mem(-CF)

SUB/SBB mem,imm/reg #mem←mem-imm/reg(-CF)

- 减量指令 DEC

DEC reg/mem # reg/mem←reg/mem-1

- 求补指令 NEG

NEG reg/mem #reg/mem←0—reg/mem

- 比较指令 CMP

SUB reg,imm/reg/mem; reg-imm/reg/mem

SUB mem,imm/reg; mem-imm/reg

- 除DEC不影响CF标志外
- 其他按定义影响全部状态标志位

〔例4-8〕：sub指令对标志位的影响



mov al, 0x80

sub al, 0x7 #al=0x80(-128)−0x7=0x79(121), 结果错误

#EFLAGS=0xa12, OF=1, CF=0, ZF=0, SF=0, PF=0, AF=1

mov eax, 0x80

sub eax, 0x7 #eax=0x00000080−0x00000007=0x00000079

#EFLAGS=0x212, OF=0, CF=0, ZF=0, SF=0, PF=0, AF=1

〔例4-10〕：64位二进制减法运算



```
.data
dvar1: .long 0x82347856,0x67783000
dvar2: .long 0x12348998,0x67762000
dvar3: .quad 0
.text
mov rbx,0
lea r8, dvar1[rip]
lea r9, dvar2[rip]
lea r10, dvar3[rip]
```

〔例4-10〕：64位二进制减法运算



```
mov eax,[r8]
```

```
sub eax,[r9]
```

#低位双字相减

#eax=0x6fffeebe,EFLAGS=0x00000A16

```
mov [r10],eax
```

```
mov eax,[r8+4]
```

```
sbb eax,[r9+4]
```

#高位双字相减，再减CF，

#eax=0x00021000,EFLAGS=0x00000206

```
mov [r10+4],eax
```

```
mov rax,dvar3[rip]
```

.....

大小写字母转换



大写=小写-0x20
小写=大写+0x20

mov al,' A' #al=0x41 (' A' 的ASCII码)

add al, 0x20 #al=0x61 (' a' 的ASCII码)

mov al,' a' #al=0x61 (' a' 的ASCII码)

sub al,0x20 #al=0x41 (' A' 的ASCII码)

求补指令NEG



- 可用于对负数求补码或由补码求其绝对值

mov ax,0xff64

neg al #AX=0xFF9C, OF=0, SF=1, ZF=0, PF=1, CF=1

sub al,0x9d #AX=0xFFFF, OF=0, SF=1, ZF=0, PF=1, CF=1

neg ax #AX=0x0001, OF=0, SF=0, ZF=0, PF=0, CF=1

dec al #AX=0x0000, OF=0, SF=0, ZF=1, PF=1, CF=1

neg ax #AX=0x0000, OF=0, SF=0, ZF=1, PF=1, CF=0

乘法指令



- 无符号数乘法指令 **MUL**
- 有符号数乘法指令 **IMUL**
- 计算二进制数乘法: $0xA5 \times 0x64$
 - 用MUL指令作无符号数乘法:

$0x4074 (= 16500) = 0xA5 (= 165) \times 0x64 (= 100)$

- 用IMUL指令作有符号数乘法:

$0xDC74 (= -9100) = 0xA5 (= -91) \times 0x64 (= 100)$

乘法指令



指令类型	操作数组合及功能	举例
无符号数乘法 MUL src	$AX = AL \times r8/m8$ $DX.AX = AX \times r16/m16$ $EDX.EAX = EAX \times r32/m32$ $RDX.RAX = RAX \times r64/m64$	mul bl imul bx mul dword ptr var[rip] imul dword ptr var[rip] mul qword ptr var[rip] imul qword ptr var[rip]
有符号数乘法 IMUL src		
双操作数乘法 IMUL dest,src	$r16 = r16 \times r16/m16/i8/i16$ $r32 = r32 \times r32/m32/i8/i32$ $r64 = r64 \times r64/m64/i8/i64$	imul eax,10 imul ebx,ecx imul rbx,rcx
三操作数乘法 IMUL dest,src,imm	$r16 = r16/m16 \times i8/i16$ $r32 = r32/m32 \times i8/i32$ $r32 = r64 \times r64/m64/i8/i64$	imul ax,bx,-2 imul eax,dword ptr [rsi+8],5 imul rax,qword ptr [rsi+8],5

- 无符号除法指令 DIV
- 有符号除法指令 IDIV
- 除法指令可能产生除法溢出
 - 对DIV指令，除数为0，或者在字节除时商超过8位，在字除时商超过16位，或者双字除时超过32位，则发生除法溢出
 - 对IDIV指令，除数为0，或者在字节除时商不在 - 128 ~ 127 范围内，在字除时商不在 - 32768 ~ 32767 范围内，或者在双字除时商不在 - 2^{31} ~ $2^{31} - 1$ 范围内，则发生除法溢出
- 除法错溢出，将产生编号为0的内部中断

除法指令



指令	操作数组合及功能	举例
无符号除法: <code>DIV src</code>	$AL \leftarrow AX \div r8/m8$ 的商 $AH \leftarrow AX \div r8/m8$ 的余数 $AX \leftarrow DX$. $AX \div r16/m16$ 的商	<code>div bl</code> <code>idiv bl</code> <code>div bx</code>
有符号除法: <code>IDIV src</code>	$DX \leftarrow DX$. $AX \div r16/m16$ 的余数 $EAX \leftarrow EDX$. $EAX \div r32/m32$ 的商 $EDX \leftarrow EDX$. $EAX \div r32/m32$ 的余数 $RAX \leftarrow RDX$. $RAX \div r64/m64$ 的商 $RDX \leftarrow RDX$. $RAX \div r64/m64$ 的余数	<code>idiv bx</code> <code>div ebx</code> <code>idiv ebx</code> <code>div rbx</code> <code>idiv rbx</code>

〔例4-14a〕：无符号数除法



```
void eg0414a(unsigned long x, unsigned long y,  
unsigned long *qp, unsigned long *rp)  
{  
    unsigned long q=x/y;  
    unsigned long r=x%y;  
    *qp=q;  
    *rp=r;}
```



```
# ecx=x,  edx=y,  r8=qp,  r9=rp  
mov  eax, ecx  
mov  r10d, edx          #y保存到r10d, 为什么?  
mov  edx, 0             #被除数edx.eax  
div  r10d  
mov  DWORD PTR [r8], eax #eax=商  
mov  DWORD PTR [r9], edx #edx=余数
```

〔例4-14b〕：有符号数除法



```
void eg0414b(long x, long y, long *qp, long *rp){  
    long q=x/y;  
    long r=x%y;  
    *qp=q;  
    *rp=r;}  
↓
```

```
# ecx=x,  edx=y,  r8=qp,  r9=rp  
mov  eax, ecx  
mov  r10d, edx          #y保存到r10d  
cdq                      # eax符号扩展到edx.eax  
idiv  r10d  
mov  DWORD PTR [r8], eax  #eax=商  
mov  DWORD PTR [r9], edx  #edx=余数
```

零位扩展和符号扩展指令

- 零位扩展对应无符号数：MOVZX指令
 - 前面加0实现位数扩展
 - 0x80：8位无符号数，零位扩展为16位：0x0080
- 符号扩展对应有符号数：MOVSX指令
 - 前面加符号位（最高位）实现位数扩展
 - 0x64：8位有符号数，符号扩展成16位：0x0064
 - 0xFF00：16位有符号数据
 - 符号扩展成32位：0xFFFFFFFF00，都表达真值：-256
 - 真值-1，字节量补码：0xFF
 - 字量补码：0xFFFF，双字量补码：0xFFFFFFFF

 位数加长，大小没变

零位扩展和符号扩展指令



指令类型	指令	举例
零位扩展	MOVZX r16,r8/m8	movzx di,bvar[rip]
	MOVZX r32,r8/m8/r16/m16	movzx eax,ax
	MOVZX r64,r8/m8/r16/m16	movzx rax,bl
符号扩展	MOVSX r16,r8/m8	movsx ax,al
	MOVSX r32,r8/m8/r16/m16	movsx edx,bx
	MOVSX r64,r8/m8/r16/m16	movsx rax,bl
	MOVSXD r64, r32/m32	movsxd rax, ebx

零位扩展和符号扩展指令示例



mov al,0x82	/*al=0x82*/
movzx bx,al	/*bx=0x0082*/
movzx ebx,al	/*ebx=0x00000082*/
mov cx, 0x1000	/*cx=0x1000*/
movzx edx,cx	/*edx=0x00001000*/
mov al,0x82	/*al=0x82*/
movsx bx,al	/*bx=0xFF82*/
movsx ebx,al	/*ebx=0xFFFFFFFF82*/
mov cx,0x1000	/*cx=0x1000*/
movsx edx,cx	/*edx=0x00001000*/

除法溢出



- 除法指令可能产生除法溢出
 - 对DIV指令，除数为0，或者在字节除时商超过8位，在字除时商超过16位，在双字除时超过32位，或四字除时超过64位则发生除法溢出。
 - 对IDIV指令，除数为0，或者在字节除时商不在 - 128 ~ 127 范围内，在字除时商不在 - 32768 ~ 32767 范围内，在双字除时商不在 - 2^{31} ~ $2^{31} - 1$ 范围内，或者四字除时商不在 - 2^{63} ~ $2^{63} - 1$ 范围内则发生除法溢出。
- 除法错溢出，将产生编号为0的内部中断

```
mov ax, 20000
```

```
mov bl, 10
```

```
div bl      #20000 ÷ 10 = 2000, 商在AL中放不下, 产生溢出
```

```
#用16位除法避免发生溢出:
```

```
mov dx, 0
```

```
mov ax, 20000
```

```
mov bx, 10
```

```
div bx      #20000 ÷ 10 = 2000, 商在AX中可以放下, 不产生溢出
```